

TravelJournal

An offline-first Android app for planning trips, running a day-by-day itinerary, tracking spend and keeping a travel journal.

STUDENTS	Anant Sharma and Eimun Akit Purti
TRACK	Application Development
MENTOR	Rohit Gupta
PLATFORM	Android, API 30 and above
STACK	React Native (Expo) · JavaScript · expo-sqlite · react-native-maps · Axios · OpenWeather API
TIMELINE	12 Weeks
DOCUMENT	Version 1.0, 02 September 2026
GITHUB	github.com/DarkBird10020/traveljournal

Table of content

1. Business Requirements Document (BRD)	4
1.1 Business Context	4
1.2 Business Objectives	4
1.3 Stakeholders	4
1.4 Scope	5
In Scope	5
Out of Scope	5
1.5 Business Requirements	6
1.6 Assumptions	6
1.7 Constraints	7
1.8 Risks	7
2. User Stories	8
Epic 1 – Trip Management	8
Epic 2 – Itinerary Planning	8
Epic 3 – Map and Places	9
Epic 4 – Weather and Packing	9
Epic 5 – Budget and Expenses	10
Epic 6 – Journal and Documents	10
Epic 7 – Offline Reliability	11
Epic 8 – Reminders	11
3. Technical Requirements Document (TRD)	12
3.1 Project Overview	12
3.2 Core Learning Objectives	12
3.3 Must-Have Requirements	12
3.4 Good-to-Have / Stretch Requirements	13
3.5 Non-Functional Requirements	13
3.6 Technical Requirements Traceability	14
3.7 Technology Rationale	14
3.8 Final Deliverables	15
3.9 Project Structure	15
4. High-Level Design (HLD)	16
4.1 System Architecture Overview	16
4.2 Architecture Diagram	16
4.3 Component Responsibilities	17
4.4 Data and Network Layer	17
4.5 End-to-End Flow	18
5. Low-Level Design (LLD)	19
5.1 Repository Structure	19

5.2 Key Interfaces	20
5.3 Core Algorithms	20
5.4 Cache-First Fetch Flow	21
5.5 Reminder Scheduling Flow	21
5.6 Weekly Implementation Roadmap	21
5.7 Design Patterns	22
5.8 Validation and Error Handling	22
6. API Specification	23
6.1 External APIs	23
6.2 Repository Contracts	23
6.3 Service Contracts	24
6.4 Standard Errors	24
7. Database / Data Design	25
7.1 Core Tables	25
7.2 ER-Style Relationships	25
7.3 Indexes	26
7.4 Data Lifecycle	26
8. UX Requirements — Android App	27
8.1 Information Architecture	27
8.2 Primary User Flow	27
8.3 Key Screen Requirements	27
8.4 States	28
8.5 Visual Design Direction	28
8.6 Accessibility	28
9. Security Design	29
9.1 Authentication	29
9.2 Local Data Protection	29
9.3 Permissions	29
9.4 API Key Management	29
9.5 Transport Security	30
9.6 Privacy Rules	30
9.7 Important Product Rule	30
10. Testing Strategy	31
10.1 Test Levels	31
10.2 Sample Test Cases	31
10.3 Offline Conformance Checklist	32
10.4 Performance Benchmarking	32
10.5 Device Matrix	32
10.6 Definition of Done for a Feature	33
10.7 Evaluation Readiness	33

- 11. CI/CD 34
 - 11.1 Pipeline 34
 - 11.2 Branching Strategy 34
 - 11.3 Pull Request Requirements 35
 - 11.4 Quality Gates 35
 - 11.5 Release Process 35
 - 11.6 Rollback 35
- 12. Observability 36
 - 12.1 Logs 36
 - 12.2 Metrics 36
 - 12.3 Crash and Error Reporting 36
 - 12.4 In-App Diagnostics 37
 - 12.5 Post-Release Review 37
- 13. Deployment Architecture 38
 - 13.1 Environments 38
 - 13.2 Build Profiles 38
 - 13.3 Publishing Targets 38
 - 13.4 Rollback 39
 - 13.5 Configuration Management 39
 - 13.6 Responsibilities 39
- 14. Glossary 40

1. Business Requirements Document (BRD)

1.1 Business Context

Trip plans today are scattered across screenshots, chat threads, notes apps and booking emails, and almost none of it is usable once mobile data stops. Most travel apps are built online-first and revolve around bookings, so they assume connectivity and treat the traveller's own plan as secondary. Roaming abroad is expensive and coverage is unreliable, which means a cloud-only itinerary is effectively unavailable at the exact moment it is needed — and the cost of that is missed reservations, wasted paid activities, overspending with no visibility of the budget, and memories that never get recorded while they are fresh. TravelJournal keeps the whole trip in one Android app — itinerary, saved places on a map, weather, budget, documents and a photo journal — and keeps every part of the personal trip data readable with no network connection.

1.2 Business Objectives

ID	OBJECTIVE	SUCCESS SIGNAL
OB-01	Keep the entire trip plan in one place	Itinerary, map, budget and journal all reachable within two taps of Home
OB-02	Work with no internet connection	Every trip screen renders correctly in airplane mode
OB-03	Make planning fast enough that people actually do it	First trip created and first activity added in under 30 seconds
OB-04	Help travellers stay inside a budget	Live remaining budget and category breakdown for every trip
OB-05	Capture memories while they are fresh	A photo or note can be attached to any trip day in three taps
OB-06	Deliver a finished product, not a demo shell	Signed release build, unit tests above 80 percent on core logic, documented repository
OB-07	Respect privacy by default	No account needed and no personal content leaves the device

1.3 Stakeholders

STAKEHOLDER	INTEREST
Student engineering team (Anant Sharma, Eimun Akit Purti)	Build and ship a production-grade offline-first Android application
Independent traveller	Plans a trip alone or as a couple, travels abroad without a data pack, needs the plan to be reliable offline
Student group trip	Fixed budget, shared itinerary, wants visible spend by category and a simple shared plan
Weekend and short-trip planner	Plans in a few minutes on the phone, values speed over depth, mostly uses today's plan view
Journal keeper	Records the trip while it happens with photos and short notes tied to the day they belong to
Mentor (Rohit Gupta)	Weekly checkpoints, architecture review, a working installable build on his own device
Evaluators / reviewers	A working system, clear documentation, test evidence and a recorded demo

1.4 Scope

In Scope

- Local login and traveller profile with a hashed PIN
- Trip create, edit, archive and delete, with day sheets generated from the date range
- Activity CRUD with a day timeline view and clash warnings
- Google Maps with trip pins, place search and saved places
- Weather with a five-day forecast, cached fallback and rule-based packing hints
- Expense tracking with categories and currency conversion on cached rates
- Photo journal with optional GPS stamp, and a documents store for tickets and bookings
- Packing checklist with templates by trip type and forecast
- Local reminders for activities and the departure day
- Offline-first SQLite storage with an offline indicator
- Material 3 theme with light and dark mode
- Signed Android build produced with EAS Build, plus README, architecture notes and a recorded demo

Out of Scope

- Hotel, flight or activity booking and payments
- Real-time collaborative editing of one trip
- Social feed, followers or public trip discovery
- Server accounts and cloud sync in the main build

- Turn-by-turn navigation
- A recommendation model trained by the team
- Wear OS companion app
- Ads, subscriptions or any commercial packaging
- Storing real card numbers or government ID numbers

1.5 Business Requirements

ID	REQUIREMENT	PRIORITY
BR-001	System shall store all trip data locally in SQLite as the single source of truth	Must-Have
BR-002	System shall let a traveller create, edit, archive and delete a trip	Must-Have
BR-003	System shall generate one day sheet per date in the trip range and hold activities against it	Must-Have
BR-004	System shall render every trip screen correctly with no network connection	Must-Have
BR-005	System shall show trip places on a map with pins coloured by day and a searchable place list	Must-Have
BR-006	System shall track a trip budget and categorised expenses with a live remaining figure	Must-Have
BR-007	System shall attach photos and notes to a trip day as journal entries	Must-Have
BR-008	System shall show current and five-day forecast weather with a cached fallback	Good-to-Have
BR-009	System shall schedule a local reminder 30 minutes before any timed activity	Good-to-Have
BR-010	System shall keep tickets, bookings and scans openable per trip with no network	Good-to-Have
BR-011	System shall suggest a packing checklist from trip length and forecast	Good-to-Have
BR-012	System shall be delivered as a signed release build installable on a mentor's device	Must-Have
BR-013	System shall keep all personal content on the device, sending out only destination coordinates and a currency pair	Must-Have

1.6 Assumptions

- Users have an Android phone on API 30 or later with about 200 MB of free storage.
- Google Maps and OpenWeather keys are available on free-tier quotas that are enough for development and the demo.

- Connectivity is available while planning, and the trip itself may be fully offline.
- Document scans stored in the app are files the user already owns and are kept only on the device.
- Two students work on the project across 12 weeks with weekly mentor checkpoints.

1.7 Constraints

CONSTRAINT	DETAIL
Time	12 weeks in phases, with the core feature list frozen after week two
Team	Two second-year students working alongside coursework
Platform	Android only, built and tested on physical devices and one low-memory emulator
Cost	Free-tier API quotas only, no paid infrastructure
Architecture	The local database is the single source of truth and the network is an enhancement, never a dependency

1.8 Risks

The hardest part of the build is the offline-first data layer. Keeping SQLite as the single source of truth while weather, geocoding and currency data depend on the network means designing a cache-first service layer with freshness rules, write-through updates and clean fallbacks, and keeping the interface honest about what is live and what is cached. The next two hardest are the Google Maps setup with release signing, and keeping scheduled reminders correct when activities are edited or deleted.

RISK	LEVEL	MITIGATION
Maps key misconfigured or quota exceeded	High	Restrict the key to the release certificate, cache geocoding results, set quota alerts, keep a static map fallback
Weather API rate limits	Medium	Three-hour cache per destination and fetch only on trip open or pull to refresh
Permission changes on newer Android versions	Medium	Request at point of use with a reason, detect denial and degrade the feature visibly, test on Android 13 and above
Photo and document storage growth	Medium	Compress on import, keep URIs rather than blobs, show storage per trip with a cleanup action
Scope creep from stretch features	High	Freeze the core list after week two and start stretch work only in week eleven
Parallel work conflicts between two developers	Low	Feature branches, clear module ownership and peer review on every pull request

2. User Stories

Personas used below: the **Planner** who organises the trip before departure, the **Traveller** who uses the app in the field and is often offline, the **Budget Keeper** who tracks spend, and the **Journal Keeper** who records the trip as it happens.

Epic 1 — Trip Management

US-001 As a Planner, I want to create a trip with a title, destination and date range, so that I have a structure to build the plan on.

Acceptance Criteria:

- New Trip is reachable from Home and from Trips
- Title and destination are required; the end date cannot be before the start date
- The trip appears in the list at once and survives a restart
- One day sheet is created for every date in the range

US-002 As a Planner, I want to see upcoming, active and past trips in one list, so that I can switch between plans quickly.

Acceptance Criteria:

- Active trips sort first, then upcoming by start date, then archived
- Each card shows destination, date range, countdown and progress
- Tapping a card opens its itinerary; an empty list invites the user to create a first trip

US-003 As a Planner, I want to edit or delete a trip, so that the plan stays accurate when things change.

Acceptance Criteria:

- Edit is reachable from the trip header
- Changing the date range adds or removes day sheets without losing activities on the dates that remain
- Delete asks for confirmation and also removes activities, expenses, journal entries and documents
- Reminders for deleted items are cancelled

Epic 2 — Itinerary Planning

US-004 As a Planner, I want to add activities to a chosen day with a time, place and category, so that every day has a concrete plan.

Acceptance Criteria:

- Add Activity sits under every day; name is required and time, place, category and notes are optional
- A saved activity appears in the right day, sorted by time
- The place can be picked on the map or typed by hand

US-005 As a Planner, I want a warning when two activities overlap, so that I do not build an impossible day.

Acceptance Criteria:

- Overlapping time windows in the same day show a warning on both entries without blocking the save
- The warning clears when the clash is resolved, and the user can keep the overlap on purpose

US-006 As a Traveller, I want today's plan on the Home screen, so that I know what is next without navigating.

Acceptance Criteria:

- Home shows the active trip's timeline for today, the next activity with a countdown, current weather and remaining budget
- Activities that already passed are visually dimmed

Epic 3 — Map and Places

US-007 As a Planner, I want all trip places on one map, so that I can group activities that are close together.

Acceptance Criteria:

- The map draws a pin for every saved place and every located activity, coloured by day
- Tapping a pin shows the name, the day and a link to the activity
- A day filter draws the route between that day's stops in order

US-008 As a Planner, I want to search for a place and save it, so that I can collect options before fixing the schedule.

Acceptance Criteria:

- A search field sits at the top of the Map screen with a result list
- Selecting a result pans the map and drops a pin offering Save place and Add to a day
- A long press saves an unnamed point the user can label later

US-009 As a Traveller, I want to see my current position against the plan, so that I can orient myself.

Acceptance Criteria:

- My Location asks for permission at the moment it is used
- When granted, the map centres on the device with a distinct marker; when denied, a short message explains what is unavailable
- Location is never tracked in the background

Epic 4 — Weather and Packing

US-010 As a Planner, I want current and forecast weather for the destination, so that I can plan outdoor time and pack correctly.

Acceptance Criteria:

- The trip screen shows temperature, condition and city with a five-day strip
- Data is fetched when the trip opens and on pull to refresh
- Offline it shows the cached reading with a last-updated time
- A failed call shows "Weather unavailable" and never blocks the screen

US-011 As a Planner, I want a packing list suggested from my trip, so that I do not forget essentials.

Acceptance Criteria:

- The list is seeded from trip length and forecast, such as rain, cold or beach
- Items can be ticked, edited and added
- Completion percentage shows on the trip card

Epic 5 — Budget and Expenses

US-012 As a Budget Keeper, I want to set a trip budget and log expenses against it, so that I always know what is left.

Acceptance Criteria:

- Budget and home currency are set at creation or later
- An expense records amount, currency, category, date and an optional note
- Remaining budget updates at once; going over budget shows a clear state rather than a blocking error

US-013 As a Budget Keeper, I want to see where the money went, so that I can adjust the rest of the trip.

Acceptance Criteria:

- A category chart and a per-day spend list are shown
- Foreign currency expenses are converted on cached rates and labelled with the rate date
- Totals recompute correctly with no network

Epic 6 — Journal and Documents

US-014 As a Journal Keeper, I want to attach photos and notes to a trip day, so that the trip is recorded as it happens.

Acceptance Criteria:

- An entry can be created from the day sheet or the Journal tab
- Photos are taken or picked, compressed and stored on the device
- A GPS stamp is added when permission allows
- The journal shows a timeline per trip and works fully offline

US-015 As a Traveller, I want tickets and bookings kept in the app, so that I can show them with no network.

Acceptance Criteria:

- PDFs and images can be attached per trip with a label

- Documents open inside the app with no connectivity
- File size and total storage are visible; deleting a document also deletes the local file

Epic 7 — Offline Reliability

US-016 As a Traveller, I want the whole plan available with no data, so that I can rely on it abroad.

Acceptance Criteria:

- Every trip screen loads from the local database in airplane mode with no blank screens, no unfinished spinner and no crash
- An offline indicator appears in the header
- Network widgets show cached data with a timestamp or a plain unavailable state

US-017 As a Traveller, I want edits made offline to be kept, so that I never lose work.

Acceptance Criteria:

- Every write goes to the local database first, whatever the connection state
- No write waits on a network call
- Data survives force stop and a device restart

Epic 8 — Reminders

US-018 As a Traveller, I want a reminder before each activity, so that I do not miss anything I planned.

Acceptance Criteria:

- A local notification is scheduled 30 minutes before any timed activity and names the activity and its place
- Tapping it opens that day's itinerary
- Editing the time reschedules it and deleting the activity cancels it

US-019 As a Planner, I want a nudge before departure, so that packing and checks are finished in time.

Acceptance Criteria:

- A reminder the evening before departure lists unchecked packing items
- It is suppressed when the checklist is complete
- No reminders are scheduled for archived trips

3. Technical Requirements Document (TRD)

3.1 Project Overview

This document covers the full design and implementation plan for TravelJournal, an offline-first Android application built as a single React Native (Expo) codebase. The goal is to go beyond a prototype: every layer from the SQLite schema to the cache-first network services is built to production standards, tested on real devices in airplane mode, and delivered as a signed release build with documentation and a recorded demo.

3.2 Core Learning Objectives

DOMAIN	WHAT WE BUILD TOWARDS
Mobile Engineering	Building a multi-screen Android app with navigation, theming and platform permissions in React Native
Offline-First Architecture	Designing a local database as the single source of truth with cache-first services and honest live-versus-cached UI
Data Modelling	A normalised SQLite schema with migrations, indexes, transactions and cascade deletes
Platform Integration	Maps, geolocation, camera, file system and local notifications through Expo modules
Performance Engineering	Cold-start budgets, list virtualisation and map memory behaviour measured on a low-memory device
Release Engineering	Producing a signed, installable Android package with EAS Build and managed secrets

3.3 Must-Have Requirements

- Local login and profile with a hashed PIN and a session that survives restart
- Trip management: create, edit, archive, delete, with cover photo, destination, dates and progress state
- Day-by-day itinerary: day sheets from the date range, activities with time, place, category and notes, timeline view and reorder
- Map and saved places: trip pins coloured by day, place search, long-press save, route line between one day's stops
- Budget and expenses: trip budget, categorised expenses, conversion on cached rates, spend chart and over-budget state
- Photo journal: photos and notes attached to a trip day with an optional GPS stamp
- Offline storage: all trip data in SQLite, media on the app file system, offline indicator, no blank screens in airplane mode
- App shell and theme: bottom tabs for Home, Trips, Map, Journal and Profile, Material 3 theming, dark mode, correct back behaviour

3.4 Good-to-Have / Stretch Requirements

ID	FEATURE	APPROACH AND VALUE
S-01	AI itinerary draft	Send destination, days and interests to an LLM API and turn the reply into editable draft activities the user confirms
S-02	Offline map region	Download a bounded tile set before departure so the map itself works offline, with visible storage cost
S-03	Cloud backup and restore	Node and Express service with MongoDB, user-triggered sync, last write wins per record
S-04	Trip sharing and split	Export a trip as a JSON file a co-traveller can import, plus a per-person expense split summary
S-05	Trip recap PDF	One-tap recap of itinerary, spend and photos rendered with expo-print, useful as a demo artefact
S-06	Voice journal notes	Short audio memories stored with a journal entry for moments when typing is impractical
S-07	Home screen widget	Android widget with the next activity and days to departure for the active trip

3.5 Non-Functional Requirements

ID	ATTRIBUTE	TARGET
NFR-01	Startup	Cold start to an interactive Home screen in under 3 seconds on a 4 GB RAM phone
NFR-02	Smoothness	60 fps scrolling with more than 200 activities and no stutter on map pan and zoom
NFR-03	Offline availability	All trip, itinerary, budget, journal and document screens usable with no network
NFR-04	Network behaviour	API responses under 2 seconds on mobile data, 8-second timeout, then cached fallback
NFR-05	Reliability	No crashes on the main flows across the device matrix and no application-not-responding dialogs
NFR-06	Storage	Install under 60 MB, photos compressed to roughly 300 KB, per-trip usage shown to the user
NFR-07	Accessibility	Touch targets of at least 48 dp, WCAG AA text contrast, TalkBack labels on every control
NFR-08	Maintainability	ESLint and Prettier clean, no file above 300 lines, above 80 percent unit coverage on core logic
NFR-09	Privacy	Nothing but destination coordinates leaves the device, and no personal content is logged

3.6 Technical Requirements Traceability

OBJECTIVE	FEATURES	USER STORIES	VERIFIED BY
OB-01	F-02, F-03, F-04, F-06, F-07	US-001 to US-006, US-012, US-014	Navigation test, user acceptance test
OB-02	F-05, F-08, F-10	US-010, US-016, US-017	Airplane mode checklist per screen
OB-03	F-02, F-03, F-12	US-001, US-004, US-006	Timed task test with five users
OB-04	F-06	US-012, US-013	Unit tests on totals and conversion
OB-05	F-07, F-10	US-014, US-015	Tap count review, media tests
OB-06	All core features	All	Coverage report, signed build, README
OB-07	F-01, F-08, F-10	US-016, US-017	Network traffic inspection, code review

Feature identifiers referenced above: F-01 Login and profile · F-02 Trip management · F-03 Day-by-day itinerary · F-04 Map and saved places · F-05 Weather and packing hints · F-06 Budget and expenses · F-07 Photo journal · F-08 Offline storage · F-09 Reminders · F-10 Documents store · F-11 Packing checklist · F-12 App shell and theme.

3.7 Technology Rationale

React Native with Expo

One JavaScript codebase with managed native modules for maps, camera, notifications and the file system, which is the only realistic way for a two-person team to ship a device-quality Android app in twelve weeks. EAS Build produces the signed package without a local Android toolchain.

expo-sqlite

A real relational database on the device, so the offline-first rule is enforced by architecture rather than by discipline: joins, transactions, indexes and versioned migrations all run locally with no server.

react-native-maps

Native Google Maps rendering with marker clustering and polylines, which keeps pin-heavy trip maps smooth on a low-memory phone where a web map view would not be.

OpenWeather with Axios

A free-tier forecast API with a simple coordinate-based contract, wrapped in an Axios client with timeouts and interceptors so every call goes through the same cache-first policy.

3.8 Final Deliverables

- Signed Android release package (AAB and APK) produced with EAS Build

- Public Git repository with a clean commit history, feature branches and a README covering setup
- Architecture notes, this documentation set, and the database schema with migration history
- Unit, database and component test suites with a coverage report above 80 percent on core logic
- Completed airplane-mode checklist and device-matrix results
- Recorded demo video and the final presentation

3.9 Project Structure

```

traveljournal/
  app/
    screens/           # Home, Trips, TripDetail, Map, Journal, Profile
    components/        # Reusable UI: cards, timeline rows, chips, sheets
    navigation/        # Bottom tabs + stacks, back behaviour
    theme/             # Material 3 tokens, light and dark
  src/
    db/
      index.js         # Open, migrations, foreign keys, transaction helper
      migrations/      # Versioned schema steps
      repositories/    # tripRepo, activityRepo, placeRepo, expenseRepo,
                       # journalRepo, documentRepo, checklistRepo
    services/
      weather.js       # Cache-first forecast lookup
      geo.js           # Place search, geocoding, permanent coordinate cache
      rates.js         # Daily currency rates with a cached fallback
      notifications.js # Schedule, reschedule, cancel local reminders
      media.js         # Image pick, compress, copy into app storage
    domain/            # Pure logic: dates, overlap, budget, packing, distance
    context/           # Session, active trip, connectivity, theme providers
  tests/
    unit/             # Domain logic
    db/               # Repository and migration tests
    flows/            # Component and navigation tests
  docs/
    architecture.md
    api-reference.md

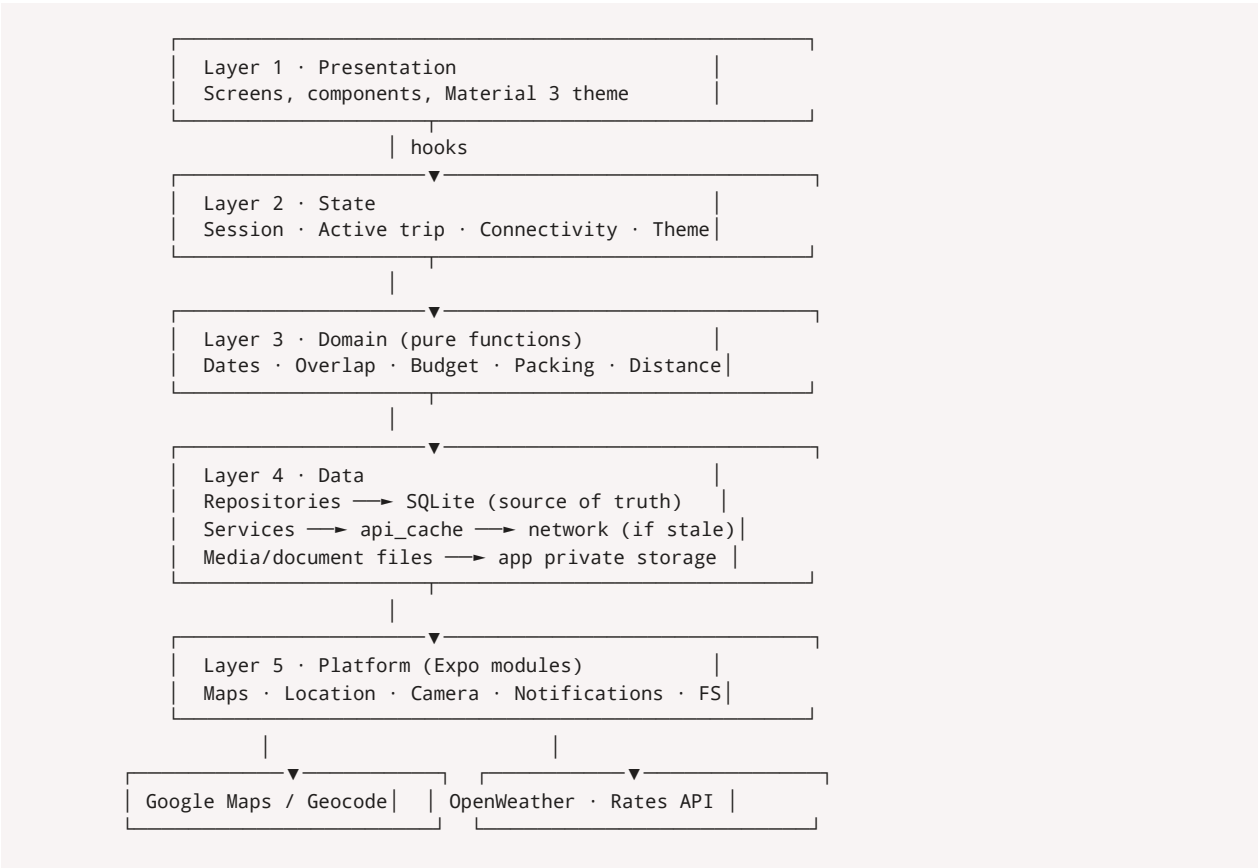
```

4. High-Level Design (HLD)

4.1 System Architecture Overview

The app is a single React Native codebase built with Expo and targeted at Android. It is layered so that screens never talk to storage or the network directly. Screens read state from context providers and hooks, hooks call repository modules, and repositories own all SQL. Anything that needs the internet goes through a service module that checks the cache first, calls the API only when the cached copy is stale, and writes the fresh response back to the cache. The local SQLite database is the single source of truth, so the app behaves the same way with or without a connection.

4.2 Architecture Diagram



4.3 Component Responsibilities

MODULE	RESPONSIBILITY
db/index.js	Opens the database, runs versioned migrations, turns on foreign keys, exposes a transaction helper
repositories/*.js	One file per entity with create, read, update, delete and query helpers; all SQL is contained here
services/weather.js	Cache-first weather lookup by coordinates with a three-hour freshness rule and a typed error result
services/geo.js	Place search and geocoding with a permanent coordinate cache, since coordinates do not change
services/notifications.js	Schedules, reschedules and cancels local reminders and keeps the identifier next to the activity
services/media.js	Picks and compresses images, copies files into app storage, reports and cleans storage usage
domain/*.js	Pure functions with no imports from React or storage, which makes them cheap to unit test
context/*.js	Session, active trip, connectivity and theme providers consumed by screens through hooks

4.4 Data and Network Layer

CONCERN	DESIGN
Source of truth	SQLite on the device. Every read a screen performs is a local read; nothing renders from a network response directly.
Write path	Local first and synchronous from the user's point of view. No write waits on connectivity, so offline edits are never queued or lost.
Read-through cache	Weather, geocoding and currency responses are written to <i>api_cache</i> with a fetch time and an expiry, keyed by endpoint and parameters.
Freshness rules	Weather three hours, currency rates one day, geocoding permanent. Offline always serves the cached copy regardless of age, labelled with its time.
Files	Photos and documents are copied into app-private storage; the database stores URIs, never blobs.
Connectivity	A single provider exposes online state to the whole app, which drives the offline badge and the decision to skip a network call entirely.

4.5 End-to-End Flow

1. **Launch.** The database opens and migrations run, the session is read, connectivity state is resolved, then Home renders from local data alone.

2. **Trip setup.** The form validates input, the destination is geocoded when online, day rows are created inside one transaction, and budget and checklist are seeded.
3. **Itinerary edits.** An activity write goes straight to SQLite, the day list is re-sorted, overlaps are checked, and a reminder is scheduled or updated.
4. **Live data.** Weather and geocoding requests read the cache, call the API only when stale and online, then write the response back with a timestamp.
5. **On-trip use.** Today's timeline, weather, remaining budget, journal entries and documents are all served from the device, with an offline badge when there is no connection.
6. **After the trip.** The trip archives itself past the end date, a spend and journal summary is generated, and the recap can be exported.

5. Low-Level Design (LLD)

5.1 Repository Structure

```
src/  
  db/  
    index.js           # openDatabase, migrate, withTransaction  
    migrations/  
      001_initial.js   # users, trips, days, activities, places  
      002_expenses.js  # expenses + budget columns on trips  
      003_journal.js   # journal_entries, documents  
      004_checklist.js # checklist_items, api_cache  
  repositories/  
    tripRepo.js        # create, listByStatus, update, remove  
    activityRepo.js    # listByDay, save, remove, reorder  
    placeRepo.js       # listByTrip, save, remove, toggleFavourite  
    expenseRepo.js     # add, listByTrip, summary  
    journalRepo.js     # listByTrip, save, remove  
    documentRepo.js    # listByTrip, attach, remove  
    checklistRepo.js   # seedForTrip, toggle, progress  
  services/  
    weather.js geo.js rates.js notifications.js media.js  
  domain/  
    dates.js           # expandRange, reconcileDays, countdown  
    overlap.js         # detectClashes  
    budget.js          # totals, byCategory, byDay, remaining  
    packing.js         # suggestItems  
    distance.js        # haversine, orderDayRoute  
  context/  
    SessionContext.js TripContext.js NetworkContext.js ThemeContext.js
```

5.2 Key Interfaces

Repository layer

tripRepo.create(trip)	-> Promise<tripId>
tripRepo.listByStatus(status)	-> Promise<TripCard[]>
tripRepo.update(id, patch)	-> Promise<void>
tripRepo.remove(id)	-> Promise<void>
activityRepo.listByDay(dayId)	-> Promise<Activity[]> // sorted, clash-flagged
activityRepo.save(activity)	-> Promise<activityId>
expenseRepo.summary(tripId)	-> Promise<BudgetSummary>
journalRepo.listByTrip(tripId)	-> Promise<JournalEntry[]>

Service layer

weather.get(lat, lng)	-> { current, forecast[], source, fetchedAt }
geo.search(query)	-> Place[] // up to 10, empty when offline
geo.reverse(lat, lng)	-> { address? , lat, lng }
rates.convert(amount, from, to)	-> { value, rateDate }
notifications.schedule(activity)	-> notificationId
media.importImage(source)	-> { uri, sizeBytes }

Domain layer

expandRange(startDate, endDate)	-> DayRow[]
reconcileDays(existingDays, range)	-> { toAdd[], toRemove[] }
detectClashes(activitiesOfOneDay)	-> activityId[]
budgetTotals(expenses, homeCcy)	-> { spent, remaining, byCategory, byDay }
suggestItems(tripLength, forecast)	-> ChecklistItem[]
haversine(a, b)	-> metres

5.3 Core Algorithms

LOGIC	HOW IT WORKS
Date range expansion	Walk from start to end date and create one day record per date, reusing existing day rows when a trip is edited
Overlap detection	Sort a day's activities by start time and compare each end time with the next start time to flag clashes
Budget aggregation	Convert every expense to the home currency on cached rates, then reduce into totals by category and by day
Distance and ordering	Haversine distance between coordinates to order the day route and estimate travel between two stops
Cache-first fetch	Return the cached value when it is fresh or the device is offline, otherwise call the API and write through
Packing rules	Map forecast conditions and trip length onto a base list, adding items for rain, cold, heat and long trips
Cascade delete	Delete a trip and its children inside one transaction, cancelling any reminders that belonged to them

5.4 Cache-First Fetch Flow

```
request(key, params, ttl)
  1. read api_cache where cache_key = key(params)
  2. if row exists and now < expires_at      -> return { data, source: "cache" }
  3. if device is offline                    -> return { data?, source: "cache", stale: true }
                                           or { error: OFFLINE } when no row exists
  4. call API with an 8 second timeout
  5. on success -> upsert api_cache(payload, fetched_at, expires_at = now + ttl)
                     return { data, source: "live" }
  6. on failure -> return cached row if present, else { error: API_TIMEOUT }
```

5.5 Reminder Scheduling Flow

```
activityRepo.save(activity)
  1. write the activity row inside a transaction
  2. if activity.notification_id exists      -> cancel it
  3. if start_time is set and in the future  -> schedule at (start_time - 30 min)
                                           store the returned id on the row
  4. on trip archive or trip delete         -> cancel every child notification id
  5. departure nudge: one notification the evening before start_date,
    suppressed when checklist progress = 100%
```

5.6 Weekly Implementation Roadmap

WEEK	DELIVERABLES	OWNER
W1-W2	Setup and design. Expo project and Android configuration, Git repository and branch strategy, SQLite schema and migrations, navigation shell with bottom tabs, wireframes and the Material 3 theme in light and dark.	Anant
W3-W4	Login and trips. Local signup and login, profile screen, trip create, edit and delete, trips list with cover photos and progress, day sheet generation.	Eimun
W5-W6	Itinerary. Activity CRUD, timeline view, time sorting, clash warnings, reorder, packing checklist with templates.	Anant
W7-W8	Map and weather. Map with trip pins and day routes, place search and long-press save, saved places, weather with a five-day forecast and the cache-first service layer.	Eimun / Anant
W9-W10	Budget, journal and reminders. Expense tracking with charts and currency conversion, photo journal, documents store, local notifications, offline indicator and a full airplane-mode pass.	Eimun / Both
W11	Stretch and polish. One or two stretch features such as the recap export or the itinerary draft, animation and empty-state polish, accessibility and contrast pass, dark mode review.	Both
W12	Testing and delivery. Test suite to target coverage, device matrix runs, bug fixing, signed release build with EAS, README and documentation, demo video and final presentation.	Both

5.7 Design Patterns

PATTERN	WHERE IT IS USED
Repository	One module owns all SQL for one entity, so screens and hooks never write queries
Layered architecture	Five strict layers with one-way dependencies; the domain layer imports neither React nor storage
Cache-aside / write-through	Every network service reads <i>api_cache</i> first and writes fresh responses back with a timestamp
Provider / context	Session, active trip, connectivity and theme are injected once and consumed through hooks
Unit of work	Trip creation and cascade delete run inside a single SQLite transaction so failures leave nothing half written
Typed result object	Services return data or a typed error object; raw exceptions never reach a screen

5.8 Validation and Error Handling

- Validation runs in the domain layer before any write: required fields, date order, non-negative amounts and valid currency codes.
- Every repository and service returns plain data or a typed error object with a code; screens map the code to a message.
- Network calls carry an 8-second timeout and fall back to cache; a failure never blocks a screen from rendering.
- Database writes that span more than one table run in a transaction and roll back as a unit.
- Migrations are versioned and idempotent: a release applies only the steps above the stored schema version.

6. API Specification

TravelJournal has no backend of its own in the main build, so this specification covers two surfaces: the third-party HTTP APIs the app consumes through its cache-first services, and the internal repository and service contracts that screens are allowed to call. Every internal function returns plain data or a typed error object and never throws a raw exception into the UI.

6.1 External APIs

ENDPOINT	PURPOSE AND CONTRACT
GET /data/2.5/weather OpenWeather	Current conditions by <i>lat</i> and <i>lon</i> . Cached for three hours under key <i>weather:{lat},{lng}</i> . Returns temperature, condition code, city name.
GET /data/2.5/forecast OpenWeather	Five-day forecast by coordinates, reduced to one entry per day for the forecast strip and the packing rules.
GET /maps/api/geocode Google	Forward geocoding for destination and place search; reverse geocoding for a long-pressed point. Cached permanently.
GET /latest Currency rates	Daily rates for the home currency. Cached for 24 hours; conversions are always labelled with the rate date used.

Common rules: HTTPS only, 8-second timeout, one retry on a network error, keys injected from EAS secrets, no request carries itinerary text, photos, documents or journal content — only coordinates and a currency pair.

6.2 Repository Contracts

FUNCTION	BEHAVIOUR
<code>tripRepo.create(trip)</code>	Validates the dates, inserts the trip, generates day rows and the checklist, returns the trip id
<code>tripRepo.listByStatus(status)</code>	Returns trips with day counts and progress, sorted for the Trips screen
<code>tripRepo.update(id, patch)</code>	Applies the patch and reconciles day rows when the date range moved
<code>tripRepo.remove(id)</code>	Cascade deletes children in a transaction, removes files and cancels reminders
<code>activityRepo.listByDay(dayId)</code>	Returns the day's activities sorted by start time with clash flags attached
<code>activityRepo.save(activity)</code>	Inserts or updates, then schedules, reschedules or clears the reminder for that activity
<code>expenseRepo.summary(tripId)</code>	Returns total spent, remaining budget, totals by category and totals by day
<code>journalRepo.listByTrip(tripId)</code>	Returns entries newest first with photo URIs resolved to local paths

6.3 Service Contracts

FUNCTION	BEHAVIOUR
<code>weather.get(lat, lng)</code>	Returns current conditions and a five-day forecast with a source flag of live or cached and the fetch time
<code>geo.search(query)</code>	Returns up to ten matches with name, address and coordinates, or an empty list when offline
<code>geo.reverse(lat, lng)</code>	Turns a long-pressed point into a readable address when possible, otherwise keeps raw coordinates
<code>rates.convert(amount, from, to)</code>	Converts using the cached daily rate and returns the value with the rate date used
<code>notifications.schedule(activity)</code>	Books a reminder 30 minutes before the start time and returns the identifier to store on the row
<code>media.importImage(source)</code>	Picks or captures a photo, compresses it, copies it into app storage and returns the local URI

6.4 Standard Errors

CODE	WHEN IT HAPPENS	WHAT THE USER SEES
VALIDATION_ERROR	A required field is missing or dates are reversed	Inline field message, the save button stays disabled
OFFLINE	A network feature is used with no connection	Cached data with a last-updated label, or a short offline note
API_TIMEOUT	An API call passes the 8-second limit	Cached value if present, otherwise a retry option
PERMISSION_DENIED	Location, camera or notification permission refused	A note on what is unavailable and a shortcut to settings
STORAGE_FULL	A photo or document import cannot be written	A message with the per-trip storage view to free space
DB_ERROR	A write or migration fails	The transaction rolls back and a retry prompt appears; nothing is half saved

7. Database / Data Design

All application state lives in one SQLite database on the device, opened with foreign keys enabled and migrated by version number. Media and documents are files in app-private storage; the database stores their URIs. The *api_cache* table is the only table holding data that originated on the network.

7.1 Core Tables

TABLE	COLUMNS
users	id, name, email, pin_hash, home_currency, theme_pref, created_at
trips	id, user_id, title, destination, lat, lng, start_date, end_date, budget, currency, cover_uri, status, created_at, updated_at
days	id, trip_id, date, day_index, note
activities	id, day_id, place_id, name, category, start_time, end_time, notes, sort_index, notification_id
places	id, trip_id, name, address, lat, lng, category, is_favourite, created_at
expenses	id, trip_id, day_id, amount, currency, amount_home, rate_date, category, note, spent_at
journal_entries	id, trip_id, day_id, text, photo_uris, lat, lng, created_at
documents	id, trip_id, label, file_uri, mime_type, size_bytes, created_at
checklist_items	id, trip_id, label, is_done, source, sort_index
api_cache	cache_key, payload_json, fetched_at, expires_at

7.2 ER-Style Relationships

```
users 1 — N trips
trips 1 — N days      days 1 — N activities
trips 1 — N places    places 1 — N activities (optional link)
trips 1 — N expenses  days 1 — N expenses (optional link)
trips 1 — N journal_entries
trips 1 — N documents
trips 1 — N checklist_items
api_cache (standalone, keyed by endpoint + parameters)
```

7.3 Indexes

INDEX	REASON
trips (user_id, status, start_date)	Trips list sorted by state and date without a full scan
days (trip_id, date)	Loading one trip's day sheets in order
activities (day_id, start_time)	Timeline query and overlap checks are already sorted
expenses (trip_id, category)	Category totals for the budget chart
journal_entries (trip_id, created_at)	Journal timeline in reverse chronological order
api_cache (cache_key) — unique	Single row per endpoint and parameter set for write-through updates

7.4 Data Lifecycle

STAGE	RULE
Create	Trip insert, day generation and checklist seeding happen inside one transaction, so a failure leaves nothing half written
Update	Editing a date range keeps day rows for dates that still exist and only adds or removes the difference
Archive	A trip flips to archived past its end date, which hides it from the active list and stops new reminders
Delete	Deleting a trip removes its days, activities, places, expenses, journal entries, documents and checklist items, deletes the related files and cancels the reminders
Cache	Weather rows expire after three hours, currency rates after a day, and geocoding results are kept because coordinates do not change
Migrations	A schema version number is stored in the database and each release applies only the steps above its current version

8. UX Requirements — Android App

8.1 Information Architecture

TAB	CONTENTS
Home	Active trip summary, today's timeline, next activity countdown, weather, remaining budget
Trips	Trip list by state, then trip detail with day sheets, budget, checklist and documents
Map	Full-screen map, search, saved places, day filter and route
Journal	Entry timeline per trip, entry composer, photo viewer
Profile	Traveller details, home currency, theme, storage usage, permissions

8.2 Primary User Flow

Home › New Trip › Trip detail with day sheets › Add Activity on day 1 › Pick a place on the map › Save › Day timeline with a reminder set

8.3 Key Screen Requirements

SCREEN	REQUIREMENTS
Home	One active trip card, today's activities with times, next activity countdown, weather chip, remaining budget, quick add button
Trips list	Cards with cover photo, destination, dates, days remaining, packing progress, swipe to archive
Trip detail	Day tabs or a scrolling day list, weather block, budget block, tabs for checklist and documents
Activity editor	Name, time pickers, place picker with map preview, category chips, notes, delete with confirmation
Map	Search bar, pins coloured by day, day filter chips, route line, my location button, save from a callout
Budget	Spent against budget ring, category chart, expense list grouped by day, add expense sheet
Journal	Entry timeline with photo thumbnails, day grouping, composer with camera and gallery, full-screen viewer
Profile	Name, home currency, theme switch, notification and location permission state, storage used per trip

8.4 States

STATE	TREATMENT
Empty	Every list has a short line explaining what will appear and one button that creates it
Loading	Local reads render immediately, so skeletons are only used for map tiles and weather
Offline	A small badge in the header, cached values labelled with the time they were fetched
Error	A short message in plain words with a retry action, never a raw error string
Over budget	The budget ring changes colour and shows the amount over, without blocking new expenses

8.5 Visual Design Direction

One typeface across the app with a clear size scale for title, section and body text. A single accent colour used for primary actions and the current day, with neutral greys for everything else. Cards with soft corners and a light divider instead of heavy shadows. Generous spacing so a day sheet reads as a list of times, not a wall of boxes. Light and dark themes from one theme object, with no hard-coded colours in screens.

8.6 Accessibility

Touch targets of at least 48 dp on every control. Text contrast that meets WCAG AA in both themes. TalkBack labels on icons, map pins and list items. Support for the system font scale up to large sizes without clipped layouts. No meaning carried by colour alone, so day colours on the map are also labelled.

9. Security Design

9.1 Authentication

The PIN is stored as a salted hash, never in plain text. The session flag lives in AsyncStorage and can be cleared from Profile. No password travels over the network because there is no server in the main build.

9.2 Local Data Protection

The database and all media sit in the app-private directory, which other apps cannot read. Documents are copied in rather than linked, so the app does not depend on external storage permissions.

9.3 Permissions

Location, camera, media and notification permissions are requested at the moment the feature is used, with a one-line reason. Denial disables only that feature. Location is never read in the background.

9.4 API Key Management

Keys are injected from EAS secrets and are not committed to the repository. The Maps key is restricted to the release signing certificate and package name, and quota alerts are enabled.

9.5 Transport Security

All calls use HTTPS with a timeout. Only destination coordinates and a currency pair are ever sent out. No itinerary text, photo, document or journal entry leaves the device.

9.6 Privacy Rules

No third-party analytics on personal content. Crash reports carry no user text. Deleting a trip deletes its files as well as its rows, and Profile offers a full reset that clears the database and media.

9.7 Important Product Rule

The device is the system of record. No feature may be designed in a way that makes a trip screen depend on a network call, and no personal content may be transmitted for any reason — including diagnostics, analytics or a stretch feature — without an explicit, user-triggered action.

10. Testing Strategy

10.1 Test Levels

LEVEL	COVERAGE AND TOOLS	TARGET
Unit	Jest on date expansion, overlap detection, budget totals, conversion, packing rules and cache policy	Above 80 percent on core logic
Database	Repository tests for CRUD, cascade deletes, migrations and survival across restart and force stop	No orphan rows or lost writes
API	Postman collections for weather and geocoding including bad key, rate limit and timeout responses	A clear message, never a crash
Component and flow	React Native Testing Library on forms, validation, navigation and Android back behaviour	All main flows pass
Offline	A written airplane-mode checklist run against every screen, plus cache freshness labels	Every screen usable offline
Performance	Cold start, screen transition and map render timings, list profiling with 200 or more activities	Under 3 seconds, 60 fps scroll
Acceptance	Five classmates plan a real three-day trip unaided while task success and confusion points are logged	First trip in under 3 minutes

10.2 Sample Test Cases

ID	CASE	EXPECTED RESULT
TC-01	Create a trip from 10 to 14 March	Five day sheets are created and dated correctly
TC-02	Set the end date before the start date	Inline validation message and the save button stays disabled
TC-03	Shorten a trip by one day after activities exist	The removed day is confirmed with the user; remaining days keep their activities
TC-04	Add two activities from 14:00 to 16:00 and 15:00 to 17:00	A clash warning appears on both entries and the save is not blocked
TC-05	Delete an activity that had a reminder	The row is removed and the scheduled notification is cancelled
TC-06	Open a trip in airplane mode	Itinerary loads from the database; weather shows the cached value with its time
TC-07	Log an expense in a foreign currency	The converted amount and the rate date are shown and totals update at once
TC-08	Deny the location permission and press my location	A short message explains what is unavailable and the map keeps working
TC-09	Force stop the app after adding a journal entry	The entry and its photo are present after relaunch
TC-10	Delete a trip with activities, expenses, photos and documents	All child rows, files and reminders are removed and no orphan data remains

10.3 Offline Conformance Checklist

- Every tab and every trip screen opened in airplane mode with no blank screen, no unfinished spinner and no crash
- Weather, map search and currency conversion each show a cached value with its fetch time, or a plain unavailable state
- Create, edit and delete performed offline for a trip, an activity, an expense and a journal entry, then verified after a restart
- Reminders scheduled offline still fire at the right time
- The offline badge appears and clears correctly as connectivity changes

10.4 Performance Benchmarking

MEASUREMENT	BUDGET
Cold start to interactive Home	Under 3 seconds on a 4 GB RAM phone
Screen transition	Under 300 ms, no dropped frames on the day timeline
List scroll with 200+ activities	60 fps sustained, virtualised list, no image thrash
Map with 50 pins	First render under 1.5 seconds, smooth pan and zoom, memory stable after ten pans
API response handling	Under 2 seconds on mobile data, 8-second timeout, then cached fallback
Install size and media	Install under 60 MB, photos roughly 300 KB after compression

10.5 Device Matrix

Two physical Android phones on different versions between Android 11 and Android 15, plus one low-memory emulator at a small screen size. Every feature is checked on all three before it is called done.

10.6 Definition of Done for a Feature

- The acceptance criteria of its user story pass
- Unit tests cover its logic and the suite is green
- It works in airplane mode where it should
- It has been checked on all three devices in the matrix
- It has no ESLint warnings and no file above 300 lines
- It has been reviewed by the other teammate
- The screen has empty, loading, offline and error states in place

10.7 Evaluation Readiness

PROOF	WHAT IS SHOWN
Demo walkthrough	Sign up, create a trip, build a two-day itinerary, save a place from the map, log an expense, add a journal photo, then switch to airplane mode and show that everything still works
Installable build	A signed release package produced with EAS Build that installs on the mentor's own device
Repository	Clean commit history, feature branches, README with setup steps, architecture notes and screenshots
Test evidence	Coverage report, Postman collection run and the completed airplane-mode checklist across the device matrix
Persistence proof	Force stop and restart showing trips, activities, expenses and journal entries intact
User feedback	Task success results and comments from five classmates who planned a real trip in the app

Targets at delivery: 100 percent of core features delivered · under 3 seconds cold start on a mid-range phone · above 80 percent unit coverage on core logic · zero crashes on the main flows.

11. CI/CD

11.1 Pipeline

```
push / pull request
1. install      npm ci
2. lint         eslint . + prettier --check
3. unit tests    jest --coverage      (fails under 80% on core logic)
4. database tests repository + migration suite on an in-memory database
5. flow tests    React Native Testing Library on the main navigation flows
6. build check   expo prebuild --no-install + type/asset validation

merge to main
7. preview build eas build --profile preview --platform android (APK)
8. smoke test    install the APK on a device, run the airplane-mode checklist

tagged release
9. release build eas build --profile production --platform android (AAB + APK)
10. artefacts    attach APK, coverage report and release notes to the tag
```

11.2 Branching Strategy

BRANCH	RULE
main	Always installable. Protected: no direct pushes, green pipeline required to merge.
feat/*	One branch per feature, owned by one teammate, named after the feature identifier (for example <i>feat/F-03-itinerary</i>).
fix/*	Bug fixes with the failing case added to the test suite in the same branch.
docs/*	Documentation and README changes only, no application code.

11.3 Pull Request Requirements

- Linked user story or feature identifier and a one-paragraph description of the change
- Screenshots or a short screen recording for any UI change, in both light and dark theme
- Tests added or updated for the logic touched
- Airplane-mode behaviour stated explicitly when the change touches data or network code
- Reviewed and approved by the other teammate before merge

11.4 Quality Gates

- Lint and format clean, with no warnings suppressed inline without a comment
- Unit coverage on the domain layer at or above 80 percent
- No file above 300 lines and no SQL outside a repository module
- Migration tests pass from the oldest supported schema version forward
- Preview build installs and launches on a real device

11.5 Release Process

1. Freeze the branch, run the full device matrix and the airplane-mode checklist

2. Bump the version and the Android version code, write release notes against the roadmap week
3. Produce the production build with EAS and archive the AAB, the APK and the coverage report
4. Install the archived APK on a clean device and re-run the demo walkthrough
5. Tag the release and update README, architecture notes and the documentation set

11.6 Rollback

- Every release is an archived, versioned package: rollback means reinstalling the previous APK, which is kept on the tag.
- Schema migrations are forward-only, so a rolled-back build ships with the migration set that matches its own schema version and never downgrades a database.
- Keys and configuration live in EAS secrets, so a rollback needs no code change to restore configuration.

12. Observability

12.1 Logs

Logging is development-only and structured: every line carries the screen, the repository or service name, the operation and a duration, plus an error code where applicable. Trip titles, notes, journal text, photo contents, document names and coordinates are never logged — only identifiers, row counts and sizes — so no personal content can leak into a log or a crash report.

12.2 Metrics

METRIC	TYPE	DESCRIPTION
cold_start_ms	Timing	Launch to interactive Home, recorded per device in the performance run
screen_render_ms	Timing	Render time per screen, labelled by screen name
db_query_ms	Timing	Duration per repository query, labelled by function
api_call_result	Counter	Network calls by service and outcome: live, cached, timeout, error
cache_hit_ratio	Gauge	Share of service reads served from <i>api_cache</i> rather than the network
offline_session_count	Counter	Sessions that ran entirely with no connectivity
notification_delivery	Counter	Reminders scheduled, fired, rescheduled and cancelled
storage_bytes_per_trip	Gauge	Media and document footprint per trip, also surfaced to the user in Profile

12.3 Crash and Error Reporting

- Crashes are captured with a stack trace, device model, Android version and app version, and nothing else.
- Typed errors returned to screens are counted by code, so the frequency of OFFLINE, API_TIMEOUT and PERMISSION_DENIED is known from testing.
- No third-party analytics SDK is added; reporting is opt-in and carries no user text.

12.4 In-App Diagnostics

Profile exposes a diagnostics view built for the mentor demo and for support: schema version, database size, storage used per trip, cache freshness per service with the last fetch time, notification permission state, and the current connectivity state. A "clear cache" action drops *api_cache* without touching trip data.

12.5 Post-Release Review

After each roadmap phase the team reviews the performance run, the error-code counts from testing, and the airplane-mode checklist results together, and records one action per finding in the next week's plan. Anything that breaks an offline guarantee is treated as a blocker, not a bug.

13. Deployment Architecture

13.1 Environments

ENVIRONMENT	PURPOSE AND CONFIGURATION
Development	Expo development build on a physical device with fast refresh, verbose logging, development API keys and a seeded sample trip
Preview	Internal APK from the <i>preview</i> profile, production-like configuration, used for the weekly mentor checkpoint and the device matrix
Production	Signed AAB and APK from the <i>production</i> profile, logging off, restricted release keys, the artefact submitted for evaluation

13.2 Build Profiles

```
eas.json
  development { developmentClient: true, distribution: "internal" }
  preview      { android: { buildType: "apk" }, distribution: "internal" }
  production   { android: { buildType: "app-bundle" }, autoIncrement: true }
```

Keys and per-profile configuration are supplied as EAS secrets and read through the app config, so no key or endpoint is committed to the repository.

13.3 Publishing Targets

- Primary: the signed APK shared directly with the mentor and evaluators, installable on their own devices
- Secondary: the AAB retained as the store-ready artefact, with the Play Store listing out of scope for this project
- Repository release tag carrying the APK, release notes and the coverage report

13.4 Rollback

The previous APK is kept on its release tag, so recovery is a reinstall rather than a rebuild. Because migrations are forward-only and the database lives on the device, a rollback ships only with migrations that match its own schema version, and user data is never rewritten by a downgrade.

13.5 Configuration Management

ITEM	WHERE IT LIVES
API keys	EAS secrets, injected at build time; Maps key restricted to the release certificate and package name
App identity	app.config.js: package name, version, version code, icons, splash, permissions
Feature flags	A single config module gating stretch features so the core build stays stable
Cache policy	One constants file holding the freshness rules, so a change is one edit and one test
Schema version	Stored in the database and asserted at launch against the migration list

13.6 Responsibilities

TASK	OWNER
Project setup, architecture, database schema and navigation shell	Anant
Login, profile and trip management screens	Eimun
Itinerary engine with activities, timeline, clash detection and reorder	Anant
Map, place search, saved places and location permissions	Eimun
Weather integration, cache-first service layer and offline mode	Anant
Budget tracker, charts, packing checklist and documents store	Eimun
Photo journal, reminders and theming in light and dark	Both
Testing, documentation, release build and final demo	Both

Support requested from the mentor

- Review of the offline-first architecture and the database schema before implementation starts
- Guidance on Google Maps key setup, restriction and quota management for Android
- Best practice for React Native performance such as list tuning, render control and map memory use
- Feedback on the interface, especially the day timeline and the dark theme
- A realistic test and coverage target for a two-person project over twelve weeks
- Help with release signing, build profiles and producing a store-ready package

14. Glossary

TERM	DEFINITION
Offline-first	The device database is the source of truth and the network only enriches it
Cache-first	Read the stored copy, call the API only when it is stale and a connection exists
Day sheet	One record per calendar date of a trip that holds that day's activities
Clash	Two activities in the same day whose time windows overlap
Repository	A module that owns all database access for one entity
Expo	A toolchain and set of native modules used to build and ship the React Native app
EAS Build	The hosted service that produces the signed Android package
Haversine	A formula for the distance between two points given their coordinates
Material 3	The Android design system the interface follows
TalkBack	The Android screen reader used for accessibility checks
Write-through	A fresh API response is written into the cache at the moment it is received
Day index	The ordinal position of a date inside a trip, used for pin colours and day filters

STUDENTS	MENTOR APPROVAL	DATE
Anant Sharma and Eimun Akit Purti	Rohit Gupta	02 / 09 / 2026